

Lista Encadeada — Organização de Computadores

<i>Luiz Augusto Scarsi</i>	<i>2311101009</i>
<i>Bruno Francisco Neckel</i>	<i>2221101035</i>
<i>Paulo Henrique Moura Feijó Braga</i>	<i>2221101020</i>

Lista Encadeada — Organização de Computadores

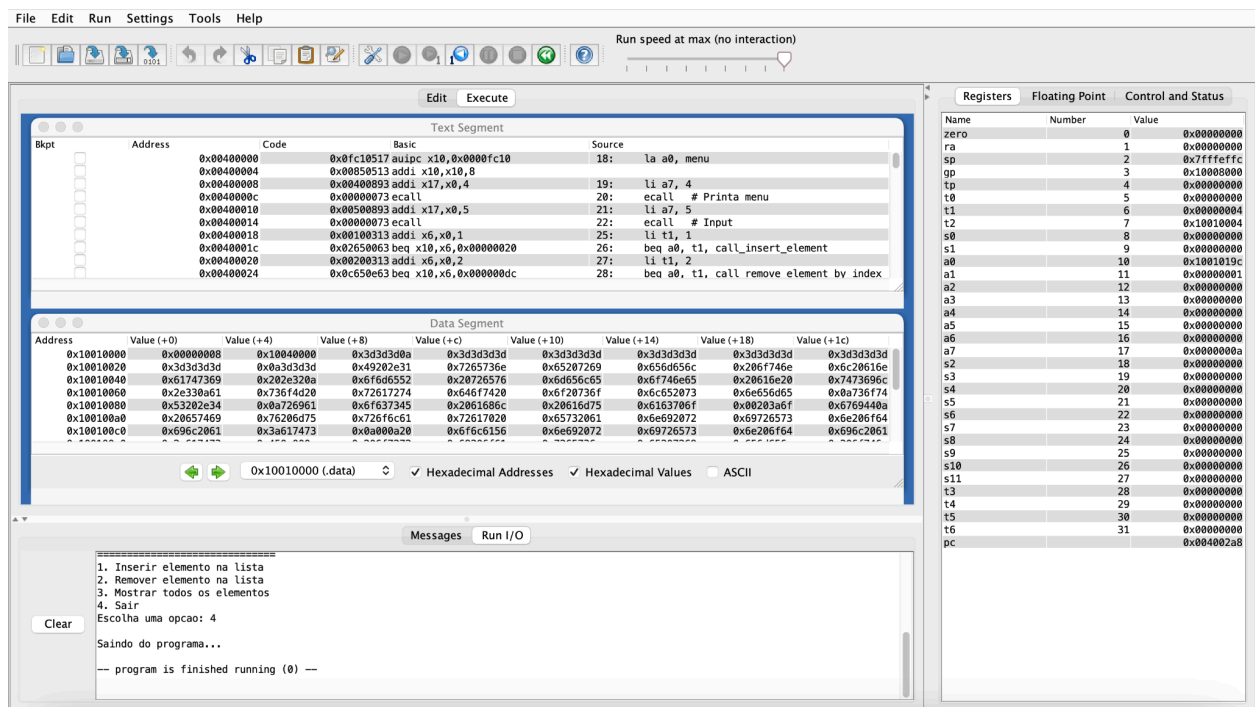
1. Introdução	1
2. Objetivo	3
3. Fundamentação Teórica	3
3.1 Lista Encadeada	3
3.2 Alocação Dinâmica de Memória	4
3.3 Uso da Stack	4
4. Desenvolvimento	5
4.1 Inserção	6
4.2 Remoção	6
4.3 Impressão da lista	7
5. Desafios e Soluções	8
6. Resultado Obtido	8
7. Conclusão	9

1. Introdução

Este trabalho apresenta a implementação de uma lista encadeada simples utilizando a arquitetura RISC-V em linguagem Assembly. A estrutura foi escolhida por permitir o armazenamento dinâmico de elementos na memória, aplicando conceitos estudados na disciplina de Organização de Computadores.

Para interação com o usuário, foi desenvolvido um menu textual contendo as principais funcionalidades da lista, permitindo selecionar operações como inserir elementos, remover elementos por índice, visualizar os valores armazenados e encerrar o programa.

Inicialmente, o desenvolvimento foi realizado utilizando o simulador online Venus. Entretanto, durante os testes foram encontrados problemas de compatibilidade, principalmente relacionados às diretivas de string e às operações de entrada de dados. O simulador não aceitava corretamente a diretiva `.asciz`, exigindo o uso de `.asciiz`, além de apresentar falhas durante a utilização de inputs. Por esse motivo, optou-se pela utilização do simulador RARS, que apresentou maior compatibilidade com as funcionalidades utilizadas no projeto. Além da implementação da estrutura de dados, o projeto também explora conceitos fundamentais da arquitetura, utilizando instruções de salto e ligação `jal` e alocação dinâmica através da syscall `sbrk`.



2. Objetivo

O objetivo principal do projeto foi implementar uma lista encadeada simples em Assembly RISC-V capaz de:

- inserir elementos dinamicamente;
- remover elementos da lista;
- percorrer a lista;
- imprimir os valores armazenados.

Além disso, o trabalho buscou aplicar corretamente:

- instruções de salto e ligação (*jal* e *jalr*);
 - uso da stack;
 - salvamento e restauração de registradores;
 - manipulação de ponteiros;
 - gerenciamento dinâmico de memória através da syscall *sbrk*.
-

3. Fundamentação Teórica

3.1 Lista Encadeada

Uma lista encadeada é uma estrutura de dados dinâmica composta por nós conectados através de ponteiros. Cada nó da estrutura implementada possui:

- Um valor inteiro;
- Um ponteiro para o próximo nó.

Cada nó ocupa 8 bytes na memória. A organização da memória utilizada no projeto foi:

Offset	Conteúdo
0	Valor inteiro

4	Ponteiro para próximo nó
---	--------------------------

3.2 Alocação Dinâmica de Memória

A criação dos nós foi realizada dinamicamente utilizando a syscall *sbrk* do RISC-V.

A syscall utilizada foi:

```
li a7, 9
li a0, node_size # (8)
ecall
```

O sistema reserva 8 bytes no heap para armazenar:

- o valor do nó;
- o ponteiro para o próximo elemento.

Esse comportamento simula o funcionamento da função *malloc* da linguagem C.

3.3 Uso da Stack

As funções implementadas utilizam a stack para:

- salvar o endereço de retorno *ra*
- preservar o fluxo correto de execução;
- manter a convenção de chamadas de função.

Exemplo utilizado:

```
addi sp, sp, -4
sw ra, 0(sp)
```

Ao final da função, o registrador é restaurado:

```
lw ra, 0(sp)
addi sp, sp, 4
```

4. Desenvolvimento

4.1 Inserção

A funcionalidade de inserção foi separada em duas funções principais: uma responsável pela interação com o menu e outra responsável pela manipulação da lista encadeada. Essa divisão foi utilizada para organizar melhor o código e separar a lógica de entrada de dados da lógica de manipulação da estrutura.

4.1.2 Chamada da Função de Inserção

A função *call_insert_element* é chamada quando o usuário seleciona a opção de inserção no menu principal.

Essa função possui como responsabilidade:

- exibir a mensagem de inserção;
- receber o valor digitado pelo usuário;
- preparar os parâmetros necessários;
- chamar a função responsável pela inserção do nó.

Antes da chamada da função principal de inserção, o endereço de retorno é salvo na stack:

```
addi sp, sp, -4  
sw ra, 0(sp)  
call insert_element
```

Esse procedimento é necessário porque a instrução de chamada sobrescreve o registrador de retorno. Após a execução da função, o valor é restaurado para garantir o retorno correto ao fluxo principal do programa.

4.1.2 Função de Inserção do nó

A função *insert_element* é responsável pela manipulação direta da lista encadeada.

Sua principal responsabilidade é:

- alocar memória dinamicamente;
- inicializar o novo nó;
- localizar o final da lista;
- realizar o encadeamento do novo elemento.

A alocação dinâmica é realizada utilizando a syscall `sbrk`:

```
li a7, 9
lw a0, node_size
ecall
```

Após a criação do nó, o programa verifica se a lista está vazia. Caso esteja, o novo nó torna-se o primeiro elemento da estrutura. Caso contrário, a lista é percorrida até o último nó para realizar o encadeamento correto.

A separação dessa rotina permitiu reutilizar a lógica da lista independentemente da interface do menu.

4.2 Remoção

Assim como na inserção, a funcionalidade de remoção foi dividida em funções separadas para organizar melhor o código e isolar a lógica de manipulação da lista.

4.2.1 Chamada da função de remoção

A função `call_remove_element_by_index`: é responsável pela interação inicial com o usuário.

Ela realiza:

- a chamada da rotina de remoção;
- o salvamento do endereço de retorno;
- o retorno ao menu principal após a operação.

4.2.2 Função de remoção

A função `remove_element_by_index` solicita ao usuário o índice do elemento que será removido e prepara os parâmetros necessários para a função principal de remoção.

Em seguida, a função *remove* realiza efetivamente a busca e reorganização da lista.

Durante a execução, a lista é percorrida sequencialmente até encontrar o índice desejado. O programa mantém referências para o nó atual e para o nó anterior, permitindo atualizar corretamente os ponteiros após a remoção.

```
mv t4, t0
mv t2, a1
li a6, 1

remove_loop:
    beq a6, t2, remove_element
    addi a6, a6, 1
    mv t4, t1
    lw t1, 4(t1)
    bnez t1, remove_loop
```

Nesse processo:

- t1 representa o nó atual;
- t4 armazena o nó anterior;
- a6 controla o índice atual da navegação.

Quando o índice desejado é encontrado, o programa chama *remove_element* que é responsável pela remoção do elemento da lista.

4.3 Impressão da lista

A impressão da lista segue o mesmo padrão de funções das anteriores

4.3.1 Chamada da impressão

A função *call_print_list* é responsável por realizar a chamada da rotina de impressão e preservar o endereço de retorno.

4.3.2 Função de impressão

A função *print_list* percorre todos os nós da lista encadeada e exibe seus valores na tela.

Inicialmente, o programa verifica se a lista está vazia. Caso contrário, os nós são percorridos sequencialmente até que o ponteiro para o próximo elemento seja nulo.

Durante o percurso, cada valor armazenado é exibido ao usuário, permitindo visualizar o conteúdo atual da lista encadeada.

5. Desafios e Soluções

Uma das principais dificuldades encontradas foi compreender o funcionamento das syscalls no simulador RARS, principalmente a utilização dos registradores *a0* e *a7* em conjunto com a instrução *ecall*. Esses elementos foram utilizados para operações como impressão de mensagens, leitura de dados do usuário e alocação de memória no heap.

Durante a implementação, foi utilizado de IA como apoio para compreender e aplicar tais partes específicas das funções de inserção, remoção e impressão da lista. Entretanto, toda a estrutura geral do programa, a organização das funções e a lógica da lista encadeada foram planejadas pela equipe.

Após a definição da estrutura principal do sistema, percebeu-se que várias partes geradas com auxílio da IA seguiam padrões que se repetiam ao longo do programa, como o salvamento e a restauração do registrador *ra* na *stack* para preservar o endereço de retorno. Dessa forma, a reutilização dessa lógica tornou a implementação das demais funções mais simples e organizada.

6. Resultado Obtido

Ao final do desenvolvimento, o programa apresentou funcionamento correto em todas as funcionalidades propostas.

A lista encadeada implementada foi capaz de realizar inserções dinâmicas de elementos, remoções por índice e impressão dos valores armazenados, mantendo o encadeamento adequado dos nós durante a execução. Além disso, as chamadas de função, o uso da *stack* e a alocação dinâmica de memória através da syscall *sbrk* funcionaram conforme esperado no simulador RARS.

7. Conclusão

O desenvolvimento do projeto permitiu compreender de forma prática conceitos fundamentais da arquitetura RISC-V, da disciplina de Organização de Computadores, e da linguagem Assembly, principalmente relacionado ao gerenciamento de memória, manipulação de ponteiros e controle de fluxo por meio de chamadas de função.

A implementação da lista encadeada contribuiu para um melhor entendimento sobre o funcionamento interno da memória em baixo nível. Além disso, o projeto também possibilitou compreender melhor a utilização de syscalls, stack e convenções de chamada.

Assim, o trabalho atingiu os objetivos propostos, demonstrando o funcionamento de uma estrutura de dados dinâmica em baixo nível utilizando Assembly RISC-V.